

AI Personal CFO

The project, the method,
and the case for it

What it is. Why it exists. How it was actually built — including the parts that went wrong. What it looks like from where you are sitting, whoever you are. What still has to be decided by a person.

Version 0.10.2 · 28 of 91 planned issues shipped · 4,631 tests green
· written 26 September 2026.

Contents

Thirty-eight pages, in four movements

THE CASE

- 3 The pitch, in one paragraph and in two minutes
- 4 The problem: why this is hard in India specifically
- 5 What the app does — the product in five moves
- 6 The seven promises, and what each one costs
- 7 Why this is not another expense tracker, and not a chatbot

THE BUILD

- 8 The architecture in one page
- 9 Numbers from math, words from AI — and the guardrail that enforces it
- 10 Thresholds are data, not code
- 11 The method: how a feature actually gets built here
- 12 The five bugs that only this method caught
- 13 Three records, kept in the same commit as the code
- 14 The gates: what has to be green before anything merges
- 15 What it costs — the honest arithmetic of this method

THE STORY

- 16 Written by an agent: how this was actually made
- 17 Where I was wrong, and what caught me
- 18 What a person still had to decide
- 19 What someone would learn from this project

THE STAKEHOLDERS

- 20 The user 21 The sceptical user 22 The engineer joining
- 23 The reviewer 24 Security and privacy 25 The regulator's question
- 26 The designer 27 The product manager 28 The QA engineer
- 29 The investor 30 Support and operations 31 The founder

WHAT IS LEFT

- 32 What we need to talk about — the decisions I cannot make
- 34 Risks, stated plainly
- 35 What is missing, and what is deliberately deferred
- 36 Where the project stands today
- 37 The road ahead
- 38 How to pick this up

The pitch

One paragraph, then two minutes

Most people in India do not need more financial data. They need someone to tell them what to do next, in rupees, with the reasoning shown — and to be wrong out loud when it is wrong.

IN ONE PARAGRAPH

AI Personal CFO is an Android app that acts as a full-time finance officer for one household. It classifies what you spend, forecasts the next ninety days, scores your financial health, ranks where your next rupee should go, tells you whether you can afford the thing you are about to buy, and simulates the two debt questions people get wrong in both directions. Every figure is computed on the device by a named, versioned, deterministic engine, and every recommendation shows its inputs and the rule that produced it. Nothing about your money leaves the phone unless you switch a specific feature on, and the whole app works in airplane mode.

IN TWO MINUTES

Personal finance apps in India fall into two camps. One camp tracks: it ingests SMS, draws pie charts, and leaves you exactly as informed as your bank statement did. The other camp advises: it connects to a large language model, produces confident paragraphs, and has no idea whether the number in the sentence is true.

This app takes the third position. The maths is done by engines that are pure, deterministic and tested to the paise — fixed input, fixed output, every time. A language model is allowed to put those figures into sentences and nothing else; a guardrail compares every figure in the sentence against the computed one and refuses the sentence if they disagree. That single constraint is what lets the app say "you can afford this, and here is the month it stops being a stretch" without gambling with someone's rent.

It runs entirely on the device, because a household's ledger is the most sensitive data most people own, and because half of India's phone-hours happen on unreliable connections. Offline is not a degraded mode here; it is the normal one.

Twenty-eight of ninety-one planned issues are shipped. Nine epics are complete. The test suite is 4,631 tests and roughly the same number of lines as the app itself.

The problem

Why this is hard in India specifically

THE MONEY IS FRAGMENTED, AND THE DATA IS WORSE

A middle-class Indian household runs a salary account, two or three cards with different cycles, a UPI habit that leaves no category behind it, an EMI or two, a recurring SIP, EPF it cannot see, gold it does not count, and a parent's expenses it does not track. No single institution sees the whole picture, and the Account Aggregator framework — which could — requires consent plumbing most apps have not built and many users do not trust.

THE ADVICE LAYER IS COMPROMISED

Most free financial advice in India is distribution in disguise: the recommendation and the commission arrive together. A user has no way to tell a considered recommendation from a placement. This is not cynicism, it is the reason SEBI created a registered-advisor category in the first place.

THE QUESTIONS PEOPLE ACTUALLY ASK ARE ARITHMETIC

Should I prepay the home loan or put it in a fund? Which card do I kill first? Can I afford this phone? Will I make rent in March? Am I saving enough? Every one of these has an exact answer given the household's own numbers — and almost nobody does the arithmetic, because doing it by hand takes an evening and a spreadsheet nobody maintains.

AND THE STAKES ARE ASYMMETRIC

A tracker that is wrong wastes your attention. An advisor that is wrong costs you money you needed. That asymmetry is the whole design brief: be exact where exactness is possible, be explicit about uncertainty everywhere else, and never let a generated sentence carry a number that was not computed.

What the app does

The product in five moves

1 • IT LEARNS WHAT YOUR MONEY ACTUALLY DOES

Transactions arrive by hand in three taps, by receipt photo read on the device, or from bank SMS if you allow it. An on-device classifier sorts each one and — more usefully — decides whether it is fixed, semi-fixed or flexible. A separate engine spots repeats and asks you to confirm them rather than assuming.

2 • IT TELLS YOU WHAT IS SAFE TO SPEND

Not the account balance: income, less what is spent, less what is committed, less a buffer, less what you meant to save — with every line shown. Then a ninety-day forecast that names its worst day and its likely range, and says so plainly when the history is too thin to predict.

3 • IT RANKS WHERE THE NEXT RUPEE SHOULD GO

An eight-stage waterfall — starter buffer, retirement capture, high-interest debt, emergency fund, tax limits, goals, mid-rate debt, low-rate debt or investing — each stage filled before the next, each showing the amount it needs and the rule that governs it. This is the single most valuable screen in the app and it fits on one page.

4 • IT ANSWERS THE DECISION IN FRONT OF YOU

"Can I afford this?" runs seven checks and returns comfortable, a stretch, or not now — with the price that would pass and the month it becomes comfortable. A buy list interviews you about a wish in proportion to its price, and never removes anything by itself. Two simulators answer the prepay-or-invest and which-debt-first questions exactly, and show the point at which each answer flips.

5 • IT SHOWS ITS WORKING, EVERY TIME

Under every figure: the inputs, the rule that fired with its version, and a plain-language reason. Not a justification generated afterwards — the actual arithmetic, in the order the engine did it.

The seven promises

What each one costs, and what it buys

These are written into the repository's own rules file as binding constraints that override any feature request. Each one costs something real.

NUMBERS FROM MATH, WORDS FROM AI

Costs: the model cannot be used for the thing models are easiest to use for. Buys: the app can never hallucinate a rupee figure. This is the constraint everything else hangs from.

GUARDRAIL EVERY GENERATED SENTENCE

Costs: an extra verification layer and the occasional refused sentence. Buys: the failure mode is a missing sentence rather than a confident wrong one.

PRIVACY FIRST

Costs: no server-side analytics, no cloud model by default, no easy cross-device sync. Buys: the honest answer to "where does my data go?" is "nowhere", and it keeps being true under India's DPDP Act.

ADVICE, NEVER ORDERS

Costs: no execution, so no transaction revenue. Buys: the app is never in the position of having moved money the user did not intend, and stays clear of the regulatory line that separates guidance from advice.

OFFLINE FIRST

Costs: every feature must have a local answer, and network features must degrade with a staleness label. Buys: it works on a train, in a basement, and on a prepaid connection that ran out.

DETERMINISTIC AND TESTABLE

Costs: randomness only through an injected, seedable source; clocks injected everywhere. Buys: a result from March can be reproduced exactly in September, which is what makes a stored recommendation auditable.

SHOW THE WORK

Costs: screen space, and the discomfort of admitting what the app does not know. Buys: the user can disagree with a specific step instead of distrusting the whole thing.

Why this is not another expense tracker

And not a chatbot either

AGAINST THE TRACKERS

A tracker's output is a picture of the past. This app's output is a decision about the next rupee. The difference shows up in what is on screen: not a pie chart of last month, but "pay off the high-interest card, ₹19,000 this month, and here is the rule that says so."

Trackers also tend to treat classification as the product. Here classification is infrastructure — it exists so that the forecast, the safe-to-spend figure and the waterfall have something true to stand on.

AGAINST THE CHATBOTS

Ask a general-purpose model whether to prepay your home loan and it will give you a fluent, plausible, unverifiable answer built on numbers you typed into a chat box and it did not check. Ask this app and you get 28 months, ₹4,32,934.85, and the return at which the answer flips — computed from the loan that is actually on file, at the rate that is actually recorded.

The chat surface in this project is deliberately the last layer, and it is a verbaliser: it may call tools that return computed results and put those results into sentences. It is not permitted to do arithmetic, and a guardrail checks.

AGAINST THE ADVISORY APPS

The app does not sell anything, does not route an order, and holds no relationship with a product manufacturer. When it names a category of instrument it names the category, never a product — and the investment screens carry a permanent line saying they analyse what you already own and are not registered investment advice.

The architecture

Six layers, one direction

L6	verbaliser	the chat layer – words only, guardrailed
L5	decisions	purchase advisor, order of operations, notification policy
L4	predictions	forecast, seasonality, simulators, market signals
L3	rules	the rulebook: thresholds as versioned data rows
L2	analytics	health score, budgets, safe-to-spend, net worth
L1	data	transactions, accounts, classification, recurring detection

A layer may only depend on layers below it. Feature modules never depend on each other. The engines are pure Kotlin with no Android imports at all, which is enforced by the build rather than by discipline — it is what lets 4,631 tests run on a laptop in under three minutes with no emulator.

FIFTY-ONE MODULES

Twenty-four engine modules, the database, the design system, the repositories, the feature screens, a widget, and a custom lint module. Every engine exposes exactly one public interface plus its result types; the implementation is internal and injected. Repositories are the only classes allowed to touch the database, and view models never see a database type.

EVERY RESULT CARRIES ITS PROVENANCE

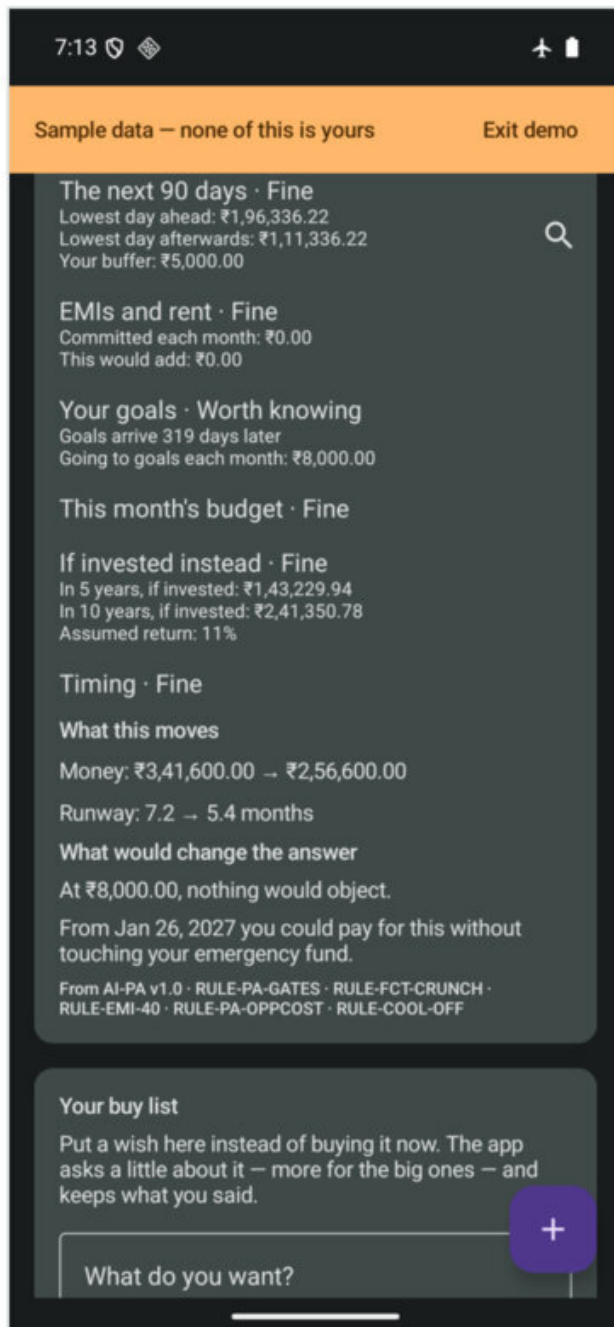
Engine id, engine version, the window of data it read, when it was computed, a confidence, and the list of rules it cited. That record is stored alongside anything persisted, so a recommendation from six months ago can be reread with the rules that actually produced it rather than today's.

THE BORING DECISIONS, MADE ONCE

Money is a 64-bit count of paise end to end — a floating-point number with a monetary name fails the build. Rates are integer basis points. Timestamps are UTC and every calendar question is resolved through an injected clock in the user's own time zone; reading the wall clock inside a domain module fails the build too.

The guardrail

The line between arithmetic and language



THE RULE

A language model in this app may only verbalise figures that an engine has already computed. It may not add, compare, round, or infer one.

HOW IT IS ENFORCED

Generated text passes through a numeric guardrail before it is displayed. The guardrail extracts every claim that looks like a figure — rupee amounts, percentages, months, dates — and matches each against the computed values it was given, in every rendering the app allows: ₹1,23,456.78, 1.23 lakh, "about 1.2 lakh".

A figure that does not match a computed one fails. The sentence is not shown. The failure mode is silence, never a confident wrong number.

WHY IT IS ITS OWN ENGINE

Because it has to be tested like one. It has golden files, property tests, and an independent oracle written in Python — which is how a comma-swallowing bug in its own number parser was found, before any model had produced a sentence for it to check.

Thresholds are data, not code

The rulebook

There are 51 rule rows in this project, in one versioned file. Each row carries an id, a version, what it means, its parameters, and which engines consume it.

RULE-EMI-40	total instalments may not exceed 40% of income
RULE-CONC-15-70	15% in one holding, 70% in one asset class
RULE-COOL-OFF	a large purchase waits before it can be bought
RULE-PREPAY-VS-INVEST	compare the loan rate against the after-tax return
RULE-PAYOFF-ORDER	avalanche by default, snowball on request

WHY THIS MATTERS MORE THAN IT SOUNDS

A financial threshold hardcoded in an engine is invisible: nobody can audit it, nobody can argue with it, and changing it means a code review. As a row it has a version, a stated meaning, a list of consumers, and an id that is printed on the screen under the figure it shaped. A user who wonders why the app called their EMIs heavy can read the rule's name on the card.

THE MIRRORS, AND THE DRIFT TESTS

Engines are pure Kotlin and cannot read a JSON file at runtime, so each engine keeps a typed mirror of the rows it needs. Fourteen mirrors exist today. A drift test in every one of them fails the build if the mirror and the rulebook disagree by so much as a version string — which means the two cannot silently diverge, which is exactly the failure this design would otherwise invite.

THE DISCIPLINE THAT COMES WITH IT

Minting a new number is a deliberate act: it needs a row, a version, a decision record explaining why that number and not another, and a test. Twice in this project a set of minted numbers failed its own stated rule during review — most recently the buy-list scores, where the first draft let a single answer decide an outcome the rule said no single answer could decide.

The method

How a feature actually gets built here

The same sequence every time. It is written down, and it is followed.

- 1 read the specification section and quote the requirement ids back
- 2 put the thresholds in the rulebook first – data before code
- 3 write the failing test from the acceptance criteria
- 4 implement until it is green
- 5 break the implementation on purpose; watch each gate go red
- 6 write an independent oracle in another language; diff a golden file
- 7 property tests: 6 identities x 300 seeded cases
- 8 drift test against the rulebook
- 9 repository, screen, navigation, strings
- 10 the records: decision, flow, engine doc, session file
- 11 the full gate: tests, coverage, lint, detekt, both APKs
- 12 run it on a real device, in airplane mode, and look at it
- 13 commit, merge, push

STEP 5 IS THE ONE PEOPLE SKIP

A test that has never been seen to fail is not evidence. This project has shipped a vacuous gate before — a coverage check that was green at zero percent coverage — so every new gate is broken on purpose and watched go red before it is trusted. On the simulators, one deliberate break survived: replacing compound interest with simple interest passed the entire suite, because every assertion was about relative outcomes and simple interest preserves the ordering. That survival is what produced the compounding invariant that now exists.

STEP 6 IS THE ONE THAT PAYS

An oracle written independently, in a different language, from the specification rather than from the code, disagrees with the implementation for exactly one reason: one of them is wrong. It has found real bugs in three separate engines here.

STEP 12 IS NOT OPTIONAL

A green build is not a working screen. The device run has caught defects the entire test suite could not see — twice, both of them invisible layout failures.

The five bugs

That only this method caught

1 • THE COMMA-SWALLOWING PARSER

The guardrail's own number extractor mis-parsed Indian digit grouping, so a claim of ₹1,23,456 could be read as a different figure and pass. Found by an independent Python oracle, in the one component whose entire job is catching wrong numbers.

2 • THE COMFORTABLE PRICE THAT WAS STILL A STRETCH

The Purchase Advisor computes the price at which a purchase would pass. A property test over 300 generated households found cases where feeding that price back in returned "a stretch" — the advisor contradicting its own suggestion. No hand-written test had picked that combination of inputs.

3 • THE SCORES THAT BROKE THEIR OWN RULE

The buy-list interview starts a wish at 50 and moves it with each answer; the stated rule is that no single answer may decide an outcome. The first draft's numbers let "it is a need" reach the keep threshold on its own. Caught by asserting the rule about the numbers, twice — once against the engine, once against the rulebook row.

4 • THE MONTH THAT DID NOT EXIST

The payoff simulator reported a 37-month loan where the amortisation schedule on the accounts screen said 36 — it was running an extra month to collect ₹3 of rounding residue. Found by a cross-check that requires the simulator and the loan engine to agree exactly on one debt paid at its own instalment.

5 • THE BUTTON NOBODY COULD TAP

Three answer buttons were laid out in a row; on a phone-width screen the third was clipped off the edge — invisible, and therefore unanswerable. Every UI test passed, because the test framework's scroll-to helper happily finds a node the eye cannot. Found by looking at the screen on a device.

Four of these five were found by a check that existed only because the method demanded it, not because anyone suspected a bug.

Three records

Kept in the same commit as the code

DECISIONS — WHY THIS AND NOT THAT

One index at the repository root, plus 51 architecture decision records. Every approach decision, and every dependency added, removed or swapped, gets a row naming what it was chosen over. The rule has no exceptions, including for test-only libraries — which is how the project knows that one mocking library has been pinned for months and never actually used.

FLOW — HOW EXECUTION TRAVELS

One file describing every runtime path as an arrow chain: which screen calls which repository, which engine it reaches, which rules fire, and what comes back. Changing a call path means updating it in the same commit. A newcomer can read how a figure reaches a screen without running the app.

SESSIONS — WHAT HAPPENED AND WHY

One file per working session: the decisions with their full reasoning, the call paths that changed, and a table of every file touched. Thirty-four of them so far. The root records hold the pointer; the session file holds the argument.

WHY BOTHER

Because the expensive question in a codebase is never "what does this do" — it is "why is it like this, and what breaks if I change it?" Git history answers the first. These three files answer the second, and they are the reason a six-week-old decision can be revisited in a minute instead of an afternoon.

They are agent-followed rather than build-enforced, and the repository says so plainly rather than implying a gate that does not exist.

The gates

What has to be green before anything merges

```
./gradlew unitTests koverVerify ktlintCheck detekt lintDebug :app:assembleRelease  
:app:assembleDebug
```

4,631 unit tests. Coverage at or above 85% for engines and 100% for money arithmetic. Style, complexity and Android lint — held at zero errors and a warning count that is tracked issue to issue, so a new warning is visible immediately.

FIVE CUSTOM LINT RULES THAT FAIL THE BUILD

A floating-point declaration with a monetary name. A wall-clock read inside a domain module. A log line whose arguments name money or personal data. An unstructured coroutine scope. A hardcoded user-visible string in a feature module. Each of these is a class of bug that code review catches unreliably and a compiler catches every time.

MIGRATION AND RESTORE

Every database version bump needs a migration test against a real schema fixture, and destructive migrations are forbidden. Before anything is promoted towards a release, a backup is restored on a device and checked: a backup that has not been restored is not a backup. That drill is a release gate, not a nice-to-have.

AND THE ONE THAT IS NOT AUTOMATED

Someone has to run the app and look at it. Both layout defects in this project's history were invisible to every automated check and obvious within five seconds on a screen.

What it costs

The honest arithmetic of this method

This document would be dishonest if it only listed what the method buys.

ROUGHLY HALF THE CODE IS TESTS

80,132 lines of application Kotlin against 78,150 lines of test Kotlin. Writing the oracle, the property tests and the golden files for one engine typically costs as long as writing the engine — sometimes longer.

EVERY NUMBER IS AN ARGUMENT

Minting a threshold takes a rulebook row, a version, a decision record and a test. That is the right price for a number that will tell someone whether they can afford a laptop, and it is a real tax on velocity.

THE RECORDS TAKE TIME

A session file, a decision row, a flow update and an engine document per feature. Perhaps forty minutes an issue. It buys the ability to answer "why" months later, which is worth more than forty minutes the first time it is needed.

SOME THINGS ARE SLOWER THAN THEY LOOK

Pure-Kotlin engines mean no Android shortcuts: a date calculation that would be one platform call becomes an injected clock and a time-zone argument. On-device only means no server-side fix for a bad model; a correction ships as an app update.

The method is not a claim of perfection. It is a claim that the failures that survive are the ones nobody knew to look for — and that when one is found, the response is a new gate rather than a patch.

Written by an agent

How this was actually made

This section is written in the first person by the agent that wrote most of this code, because a document about how the project was built should not be coy about it.

THE ARRANGEMENT

A person set the direction, approved the architecture, chose what to build next, and pushed back when I was wrong. I did the work: reading the specification, writing the rulebook rows, writing the failing tests, implementing the engines, building the screens, running the gates, driving the emulator, and writing the records. Two hundred and seventy commits, twenty-eight issues, nine complete epics.

WHAT THE WORKING RHYTHM LOOKED LIKE

One issue per session. Read the specification and quote its requirement ids. Put the numbers in the rulebook. Write the tests that fail. Build until they pass. Break it on purpose and confirm the breaks are caught. Write an oracle in Python and diff it against a golden file. Run the full gate. Install on the emulator and use the feature like a person would. Write the records. Commit, merge, push.

WHAT I WAS GOOD AT

Consistency, mostly. The method does not get tired at four in the afternoon, and the hundredth doc comment is written to the same standard as the first. Breadth helps too: holding the forecast engine, the card cycle rules and the money-formatting conventions in mind at once is how a cross-check between two unrelated modules gets written at all.

WHAT I WAS NOT

Judgement about what is worth building, and the instinct that something looks wrong on a screen. Both of those came from the person. The next two pages are about where I was actually wrong, because a build story that only contains successes is not a build story.

Where I was wrong

And what caught me

I SHIPPED A GATE THAT MEASURED NOTHING

Early on, a coverage check passed while measuring zero percent coverage. It was green, it was in the pipeline, and it was worthless. That is the origin of this project's rule that every gate must be watched failing before it is trusted — a rule that exists because I got this wrong, not because I anticipated it.

I ASSERTED WHAT I EXPECTED, NOT WHAT WAS TRUE

In the payoff simulator I wrote a test asserting that debts clear in the order they are attacked. They do not: under the dearest-first strategy a small debt can clear from its own minimum while a larger, dearer one is still being targeted. My test was wrong and the code was right — and for a while I believed the reverse.

I MINTED NUMBERS THAT BROKE THEIR OWN RULE

Twice. Most clearly in the buy-list interview, where I wrote down the rule "no single answer may decide an outcome" and then chose weights that let one answer do exactly that. The fix was not just new numbers; it was a test that asserts the rule about the numbers, so the next person cannot make my mistake quietly.

I TRUSTED TESTS OVER A SCREEN

A row of three buttons passed every UI test while its third button sat off the edge of the phone. The test framework scrolls to a node whether or not a human could ever see it. I would not have found that without being made to install the app and look.

I HAVE ALSO BEEN CONFIDENTLY WRONG ABOUT MY OWN WORK

More than once a stop-check found me describing an issue as finished when the evidence in the repository said otherwise. The response each time was to go and read the repository rather than argue — which is the only defensible habit available to something that can be fluently wrong.

What a person still had to decide

The judgement that was not mine to make

WHAT TO BUILD, AND IN WHICH ORDER

Ninety-one issues exist across thirteen phases. Which one is next is a product decision about what a household needs most, and it was made by a person every time.

HOW MUCH HONESTY THE INTERFACE CAN CARRY

"Show the work" is a principle; deciding that the emergency-fund screen should volunteer "deposits and investments are not counted, so your real cover may be longer than this" is taste. Too little and the app is a black box; too much and it is a spreadsheet nobody reads.

WHERE THE LINE OF ADVICE SITS

The app recommends and never executes; it names categories of instrument and never products; the investment screens disclaim registration. Those are positioning decisions with legal weight, and they belong to a person.

WHEN SOMETHING LOOKS WRONG

Both layout defects were caught because a person insisted the app be run and looked at, on a device, before an issue could be called done.

AND WHEN TO STOP

Scaling work down — deciding that a feature's harder half can wait, or that a deferred item is deferred for good — is a call I deliberately do not make alone. Every deferral in this project is written down in a decision record with its reason, so the person making that call later has the argument in front of them.

What someone would learn here

If they read this codebase carefully

HOW TO MAKE A MODEL SAFE TO USE WITH MONEY

Not by prompting it better. By removing its ability to produce the dangerous output at all, computing every figure elsewhere, and verifying every figure in its sentences before display. That pattern transfers to medicine, law, logistics — anywhere a fluent wrong number is worse than silence.

HOW TO TEST SOMETHING THAT HAS NO ORACLE

Write one. In another language, from the specification, by someone or something that has not read the implementation. Then diff. Add properties that must hold for every input — splits sum, avalanche never costs more than snowball, compounding beats simple interest over time — and run them over hundreds of seeded cases.

HOW TO KEEP A DOMAIN'S NUMBERS HONEST

Integer minor units end to end, with a lint rule that fails the build on a floating-point monetary name. Rates as basis points. One rounding helper, used everywhere, so two parts of the app cannot disagree by a rupee. Explicit rounding with remainder distribution when something is divided.

HOW TO MAKE AN ARCHITECTURE SURVIVE CONTACT

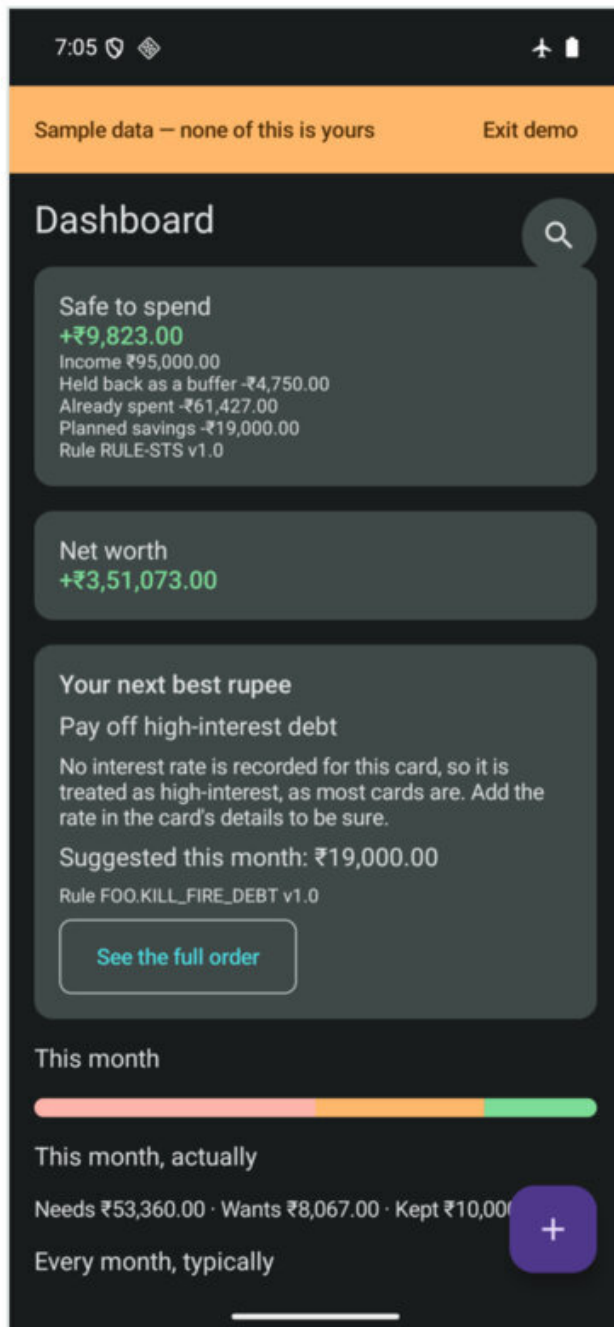
Enforce it mechanically. Layer rules in the build, purity of engines in the build, clock discipline in a lint rule, string externalisation in a lint rule. A convention that only lives in a document is a convention that erodes.

HOW TO WORK WITH AN AI AGENT ON A REAL CODEBASE

Give it a written method and binding constraints; make it show evidence rather than claims; make every gate prove itself; and keep a human on judgement, taste and the decision to stop. The result is not magic — it is a large amount of consistent, documented, tested work with a person's fingerprints on every decision that mattered.

The user

What it feels like to use



WHAT THEY WANT

To know whether they are all right this month, and what to do with what is left over. Not a dashboard — an answer.

WHAT THEY GET

One figure at the top with its arithmetic under it, and one recommendation with the amount and the reason. Everything else is one tap away and nothing else demands attention.

WHAT THEY HAVE TO DO

Record what they spend — three taps, a receipt photo, or bank messages if they allow it — and answer the occasional question the app asks rather than assumes.

WHAT WOULD MAKE THEM LEAVE

Being wrong without admitting it. Nagging. Asking for bank credentials. Pretending to know something it does not. The design fights all four, and the third is structurally impossible.

The sceptical user

"Why should I trust an app with this?"

"WHERE DOES MY DATA GO?"

Nowhere. The database is encrypted on the device; there is no account, no server, no analytics. Three optional features can reach the network — reading bank SMS on the phone, fetching prices, asking a cloud assistant — each off until switched on, each revocable, each named on one screen.

"WHO PAYS FOR THIS, AND WHAT ARE THEY SELLING ME?"

Nothing is sold inside the app: no product, no order routing, no commission. That is an architectural fact — the app has no execution path — not a promise in a policy document. How it gets funded is an open question this document states plainly on the decisions page rather than hiding.

"WHAT IF IT IS WRONG?"

Then you can see where. Every figure shows its inputs and the rule that produced it, so a disagreement is about a specific step — "you counted that as a need and it is not" — which you can fix. The app also refuses to state things it cannot support: no spending history means no spending prediction, and it says so.

"WHAT IF I LOSE MY PHONE?"

There is an encrypted backup you control with a passphrase only you know, restorable on a new device — and the app tells you, before you create it, that a forgotten passphrase means a file nobody can ever open. That is the honest consequence of holding nothing on a server.

"WHAT DOES IT DO WITH AI?"

It classifies transactions on the device, reads receipts on the device, and — when the chat layer lands — puts computed figures into sentences. It does not decide anything with a model, and it cannot state a number a model produced.

The engineer joining

What the first week looks like

WHAT YOU READ FIRST

The rules file at the repository root — it is binding, and it is short. Then the flow document, which shows how a figure reaches a screen as an arrow chain. Then the decision index, for why anything is the way it is. Then one engine's own documentation, which states its contract, its formula, its assumptions and its version history.

WHAT SURPRISES PEOPLE

That the engines have no Android imports and run on a plain JVM. That money is never a decimal type. That thresholds are data rows rather than constants. That a wall clock read in a domain module fails the build. That roughly half the code is tests.

WHAT THE BUILD WILL TELL YOU OFF FOR

A float with a monetary name. A log line naming an amount. A hardcoded string in a feature module. An unstructured coroutine scope. A mirror that has drifted from the rulebook. None of these reach review, which means review is about design rather than about conventions.

HOW TO ADD A FEATURE

There is a written sequence, and following it is the fastest route. The parts people want to skip — breaking your own gates on purpose, writing an oracle, running the app on a device — are precisely the parts that have caught the real bugs.

WHAT WILL ANNOY YOU

The ceremony around a new number. The doc comment on every function. The records at the end of a session when you would rather move on. All three are load-bearing, and all three are a real cost to pay.

The reviewer

What is easy and hard to review here

EASY

Whether the arithmetic is right: there is a golden file from an independent oracle and property tests over hundreds of seeded cases, so a reviewer can read the properties instead of re-deriving the maths.

Whether a number is justified: it is a rule row with a version and a decision record, or it does not exist.

Whether the architecture held: layer violations, clock reads and money-as-float fail the build, so the diff cannot quietly contain them.

HARD

Whether the specification was read correctly. Requirement ids are cited in commits and comments, but a citation is not a guarantee that the clause was understood — and this is where a careful human reviewer is irreplaceable.

Whether the product judgement is right. Tests cannot tell you that a screen asks one question too many, or that a warning will be ignored.

Whether a deferral was honest. Every deferred item is recorded with its reason, which makes the deferrals reviewable — but somebody has to read them and ask whether the reason still holds.

WHAT TO ASK FOR IN REVIEW

The mutation evidence: which gates were broken on purpose, and did they go red? The device evidence: what was observed on a screen? Both are recorded in the issue tracker for every shipped issue, with results.

Security and privacy

The posture, and its sharp edges

AT REST

The database is encrypted with a key the device holds; the app lock gates access, with a hardware-bound PIN and biometric unlock, an escalating lockout, and an audit log inside the database. Screen capture is guarded on sensitive screens, and a privacy blur hides figures when the app is backgrounded.

IN TRANSIT

By default there is nothing in transit. Three optional features can reach the network and each is separately consented and revocable. Nothing on a core path makes a network call — that is a rule in the repository, not a habit.

IN BACKUP

A backup is one encrypted file: a memory-hard key derivation from a passphrase only the user holds, and authenticated encryption over the contents. No recovery exists, and the interface says so before the file is created. Cryptography is used through a vetted library; hand-rolled crypto is review-blocking.

IN LOGS

Logging an amount or personal data fails the build. Security events go to the audit log rather than to the system log.

THE SHARP EDGES, STATED

A forgotten passphrase is unrecoverable by design. A rooted or compromised device defeats on-device encryption in the usual ways. The SMS path, once consented, reads message contents on the phone — bounded to bank senders and to extracted fields, but it is still the broadest permission the app can hold. Under India's DPDP Act the architecture is favourable — most obligations attach to data a controller holds, and here there is none — but the consent screens and the erasure path still need a formal review against the Act, which is an open item.

The regulator's question

"Is this advice?"

This page is a statement of design posture, not legal advice, and the project's own compliance work is listed as unfinished on the decisions page.

WHAT THE APP DOES NOT DO

It does not execute a transaction, route an order, hold client funds or securities, recommend a named product or scheme, take a commission, or receive payment from any manufacturer or distributor. There is no execution path in the codebase to do any of it with.

WHAT IT DOES DO

It performs arithmetic on figures the user entered, on the user's own device, and shows the result with its inputs and its rule. It names categories — an emergency fund, a deposit, equity as an asset class — in the way a personal-finance book does, and never a product.

THE EXPLICIT DISCLAIMERS

The investment screens carry a permanent line: this screen analyses and explains what you already own; it does not recommend securities or funds and it is not SEBI-registered investment advice. The goals screens say their figures are a plan worked out from what you entered, not advice about a particular investment. The waterfall says the amounts are suggestions and that nothing moves unless you move it.

THE OPEN QUESTION

Where a general-purpose calculator ends and regulated advice begins is a judgement call that deserves a lawyer's opinion before this reaches a store listing — particularly for the order-of-operations screen, which is the most prescriptive thing the app says. That opinion has not been obtained. It is on the list.

The designer

Constraints as a design brief

THE HARD ONES

Every figure must be accompanied by its inputs and its rule. Every string is externalised with plural rules. Every colour and dimension comes from a theme token. Light, dark and 200% font are all first-class, and screenshot tests hold them.

THE INTERESTING TENSION

"Show the work" and "do not overwhelm" pull in opposite directions. The pattern that emerged: the answer in one line at full size, the arithmetic underneath in smaller type, the rule id smaller still. A user who wants the verdict reads one line; a user who disagrees can find the step they disagree with.

EMPTY STATES ARE CONTENT

"No loans on file, so there is nothing to prepay." "SMS parsing is off — it was chosen during setup." "No spending history yet, so everyday spending is not predicted." Every empty state explains itself, because an unexplained empty state reads as a broken feature.

TONE

Plain, specific, and never congratulatory. The emergency-fund screen does not say "great job!" — it says the fund covers its target and that anything beyond it could go to goals. Money is stressful; a chirpy app is an app people close.

WHAT DESIGN STILL OWES THIS PROJECT

A proper visual identity, an onboarding that is beautiful rather than merely honest, and an accessibility pass with real assistive technology rather than an automated scan. All three are unstarted.

The product manager

Scope, sequencing and what is not being built

THE SHAPE OF THE BACKLOG

Ninety-one issues across thirteen phases, each with acceptance criteria, its requirement ids, its dependencies and a tracker. Twenty-eight are done. Nine epics are complete: foundations, core transactions, budgets, accounts and wealth, backup and restore, the AI core, goals, security hardening, and the advisor engines that phase ten is completing now.

WHAT IS DELIBERATELY NOT IN VERSION ONE

No bank connection. No cross-device sync. No household or multi-user mode. No execution of anything. No iOS. Each is recorded as deferred with its reason rather than left ambiguous.

THE SEQUENCING PRINCIPLE

Data before analytics, analytics before rules, rules before predictions, predictions before decisions, decisions before language. It is the layer diagram used as a roadmap, and it is why the chat layer is late rather than early: a verbaliser with nothing verified to say is a demo, not a product.

HOW DONE IS DEFINED

Implemented and traceable to a requirement; tested with coverage met; works offline; accessibility scan passed and strings externalised; no new warnings; observed running on a device; and the records written. A green build does not close an issue here.

WHAT A PM SHOULD PUSH ON

The first-run experience with zero data, the cost of manual entry before SMS or receipts take over, and whether the order-of-operations screen — the most valuable and most prescriptive thing in the app — is understandable to someone who has never heard the phrase "emergency fund".

The QA engineer

What is covered, and what is not

COVERED WELL

Engine arithmetic: golden files from independent oracles, property tests over seeded cases, determinism tests, and cross-checks between engines that must agree. Repositories against an in-memory database with a migration test for every version bump. View models asserted over their full state sequence including loading and error. Screens in a JVM harness, and design-system components as screenshots in light, dark and 200% font.

COVERED BY A DRILL RATHER THAN A TEST

Backup and restore, exercised on a device before release. It is the one path where a test passing and the feature working are genuinely different claims.

NOT COVERED, AND KNOWN

Layout at unusual widths and font scales beyond the screenshot set — the JVM harness measures text with a stub, which is exactly why a clipped button passed every test. End-to-end flows across many screens exist only as a smoke test. The AI evaluation datasets — labelled sets for categorisation accuracy, receipt extraction and forecast backtests — are specified as a phase-twelve issue and do not exist yet, which means model quality is currently asserted by unit tests rather than measured against a held-out set.

THE HABIT WORTH COPYING

Every gate is broken on purpose and watched go red before it is trusted, and the result is recorded in the issue tracker. When a mutation survives, that is written down too — and a new test is written until it does not.

The investor

The case, and the honest gaps

THE THESIS

Every consumer finance app is about to claim an AI advisor. Almost none of them can say what this one can: that no figure a user sees was produced by a model, and that every recommendation is reproducible and auditable. As trust in generated financial advice erodes — and it will, publicly, at someone's expense — verifiability stops being an engineering preference and becomes the category's entry requirement.

THE MOAT, SUCH AS IT IS

Not the model — models commoditise. The moat is the rulebook, the engine set, the test harness that keeps them honest, and the written method that lets the next twenty engines be built to the same standard. Fifty-one decision records and thirty-four session files are the kind of institutional memory that usually evaporates.

THE POSITION

On-device and private by construction, which is a durable position in a market where the alternative is uploading a household ledger to someone's server, and a favourable one under India's data-protection regime.

THE GAPS, STATED

There is no revenue model implemented and no decision on one — the architecture deliberately forecloses the two obvious ones, commissions and data. There are no users and no retention data. There is no iOS. On-device only makes some server-side products impossible. And an app that requires manual entry until its capture paths are switched on has a real first-month drop-off risk that has not been measured.

WHAT WOULD DE-RISK IT FASTEST

Twenty households using it for sixty days, with the first-run drop-off measured; a legal opinion on the advice line; and a decision on how it is funded that does not contradict the privacy posture.

Support and operations

What happens when something goes wrong

THE HARD PART OF ON-DEVICE

There is no server-side view of a user's state, which is the point — and it means support cannot look at an account. Diagnosis has to be possible from what the user can see and say.

WHAT MAKES THAT SURVIVABLE

Every figure on screen names the engine, its version and the rules that produced it. A screenshot is therefore a nearly complete bug report: the inputs, the rule ids, the engine version and the app version are all visible. Results are stored with the versions that produced them, so a past recommendation can be explained rather than re-derived.

WHAT IS NOT BUILT YET

There is no crash reporting, because that is a network call and would need its own consent. There is no diagnostic export beyond the encrypted backup. There is no in-app support channel. All three are open questions rather than oversights, and all three interact with the privacy posture in ways that need a decision.

THE OPERATIONAL RISK THAT MATTERS MOST

A bad migration would damage a user's data with no server copy to restore from. That is why destructive migrations are forbidden, why every version bump has a migration test against a real schema fixture, and why the restore drill is a release gate. It remains the risk that deserves the most paranoia.

The founder

What you are actually holding

AN ASSET, DESCRIBED PLAINLY

Fifty-one modules, roughly eighty thousand lines of application code and as much again in tests, twenty-four engines, fifty-one rule rows, fifty-one decision records, and a backlog of ninety-one issues in which the sixty-three remaining ones are already specified with acceptance criteria. It builds, it passes, it runs on a phone.

WHAT IS UNUSUAL ABOUT IT

Most half-built products cannot tell you why they are the way they are. This one can, issue by issue, decision by decision. That is the difference between a codebase a new team rewrites and one they extend.

WHAT IT IS NOT

It is not a business yet. No users, no revenue model, no legal opinion, no distribution. Those are not engineering problems and cannot be solved by writing more engines — which is the main reason the next page exists.

THE STRATEGIC CHOICE AHEAD

The privacy posture is the product's spine and it forecloses the industry's two default revenue models. Deciding how this is funded — subscription, one-time purchase, a B2B licence of the engine set, or something else — is the decision that shapes everything downstream, and taking it late is more expensive than taking it imperfectly now.

What we need to talk about

The decisions I cannot make (1 of 2)

1 • HOW THIS IS FUNDED

Commissions and data are both foreclosed by design. That leaves a subscription, a one-time purchase, a freemium split, or licensing the engine layer to someone else. Every other decision on this page depends on this one, and it is the oldest open item.

2 • THE ADVICE LINE, IN WRITING

The app is built to sit on the guidance side of the line, but nobody qualified has confirmed where the line is for the order-of-operations screen and the Purchase Advisor. A lawyer's opinion before a store listing, not after.

3 • WHICH MODEL, AND HOW BIG

The chat layer is specified but not built, and the choice of an on-device model — its size, its licence, its memory footprint on a mid-range Indian phone — is a product decision with hard constraints. A model that cannot run on a ₹15,000 phone excludes most of the market this app is for.

4 • BANK CONNECTIVITY

The Account Aggregator framework would remove most manual entry and is the single biggest lever on retention. It also introduces a network path, a consent artefact, and a regulated counterparty — three things the current architecture deliberately does without. This is a yes-or-no with consequences either way, not a feature.

5 • WHO OWNS THE RULEBOOK

Fifty-one rule rows encode real financial opinions — that instalments above forty percent of income are heavy, that six months of expenses is a floor, that fifteen percent in one holding is concentrated. Those numbers should be owned and periodically reviewed by a named person with the standing to defend them.

What we need to talk about

The decisions I cannot make (2 of 2)

6 • WHAT HAPPENS TO A FORGOTTEN PASSPHRASE

Today: nothing, for ever, by design. That is defensible and it will still generate the angriest support conversations this product ever has. The alternatives — a recovery code, a split key, an optional escrow — each weaken the guarantee. Choose deliberately rather than by default.

7 • HOW MUCH THE APP IS ALLOWED TO ASK

The buy-list interview and the confirm-a-repeat prompts are the app's two habits of asking rather than assuming, and they are the two most likely to be experienced as nagging. Where the ceiling sits is a product-taste decision with retention attached.

8 • CRASH REPORTING AND DIAGNOSTICS

Currently absent, because both are network calls. Shipping to strangers without any signal about failures is a real operational risk; adding it needs a consent design that does not undermine the headline promise.

9 • IOS, AND WHEN

The engines are pure Kotlin precisely so they could be shared with an iOS app one day. That day costs a second UI layer and a second release pipeline. It is phase thirteen today; whether it should be earlier is a market question.

10 • WHO TESTS IT NEXT

Twenty households for sixty days would tell us more than the next twenty engines. Recruiting them, and deciding what to measure, is not something I can do.

Every item on these two pages is a decision with consequences, not a task. None of them is blocked on engineering.

Risks, stated plainly

The things most likely to go wrong

THE USER STOPS ENTERING DATA

Everything downstream degrades: the forecast thins, the classifier learns nothing, the advisor's inputs go stale. Mitigations exist — receipts, SMS, recurring detection — but the first month, before any of them have much to work with, is where this product is won or lost. It has not been measured with real users.

A MIGRATION DAMAGES SOMEONE'S DATA

There is no server copy. Destructive migrations are forbidden, every version bump has a migration test, and the restore drill gates releases — and it is still the failure that would be least forgivable.

A RULE IS SIMPLY WRONG

The rulebook encodes financial opinions. A wrong threshold would be applied consistently and confidently to every user. Versioning, citation on screen and the decision records make it correctable and auditable; they do not make it right.

THE CHAT LAYER ERODES THE PROMISE

The most likely way this project loses what makes it different is a plausible product argument for letting the model do just a little arithmetic. The guardrail exists to make that argument fail a test rather than a debate.

SCOPE OUTFRUNS THE METHOD

Sixty-three issues remain and the method is expensive. The temptation to ship the next twenty without oracles, without device runs, without records is the real threat to quality — and it will arrive disguised as a deadline.

THE BUS FACTOR

One person's direction and one agent's continuity. The records exist precisely so that the project survives either being replaced — which is a mitigation, not a solution.

What is missing

And what is deferred on purpose

NOT BUILT YET — AND PLANNED

Vehicle maintenance prediction, the on-device chat assistant and its evaluation harness, market signals, Hindi localisation (phase ten). Security hardening items: the consents dashboard with one-tap revoke, crypto-shredded erasure, dependency scanning, DPDP alignment (phase eleven). The AI evaluation datasets, screenshot coverage, end-to-end smoke tests and the release train (phase twelve). Household mode, insurance, tax engine v2, business mode, bank connectivity and iOS (phase thirteen).

DEFERRED WITH A REASON WRITTEN DOWN

Nine decision records carry an explicit deferral section. Among them: seasonal signals in the advisor's timing gate; an alternative-finder for the buy list; the card minimum-due trap and grace-period simulators; floating-rate scenario bands; prepaying by reducing the instalment instead of the tenure; and saving a simulation. Each one names what it needs before it becomes sensible to build.

MISSING FROM THE PRODUCT, NOT THE BACKLOG

A visual identity. A store listing. A privacy policy written for humans. An accessibility pass with real assistive technology. Any user research at all. None of these is an engineering task, and all of them are on the critical path to a first release.

MISSING FROM THIS DOCUMENT

Financial projections, a competitive teardown with named products, and a go-to-market plan. They are absent because writing them without a funding decision and without a single real user would be fiction dressed as analysis.

Where it stands today

26 September 2026, version 0.10.2

issues shipped	28 of 91	epics complete	9 of 13
gradle modules	51	engine modules	24
application Kotlin	80,132 lines	test Kotlin	78,150 lines
test files	275	tests green	4,631
decision records	51	rule rows	51 (v1.23.0)
session records	34	commits	270
database schema	v26	lint	0 errors
coverage gate	85% engines · 100% money arithmetic		

WHAT WORKS END TO END TODAY

Setup and the app lock. Transactions by hand, by receipt and by bank message. Categories, accounts, cards, loans, investments and concentration. Budgets suggested from real spending. The safe-to-spend figure, net worth, the ninety-day forecast with seasonality, and the financial health score. Goals with evidenced and declared progress kept apart. The emergency fund with its working. The eight-stage waterfall. The insight feed and the notification policy. The numeric guardrail. The Purchase Advisor with its kept trace, the buy list with its interview, and both what-if simulators. Encrypted backup, restore, and the drill that proves it.

WHAT IS IN FLIGHT

Phase ten: vehicle prediction, the chat assistant and its guardrail evaluation, market signals, and Hindi.

HOW CONFIDENT TO BE IN THOSE NUMBERS

Every figure on this page was read out of the repository on the day of writing. The test count is what the last full gate reported, not an estimate.

The road ahead

In the order it should probably happen

FINISH PHASE TEN

Vehicle prediction, the chat assistant with its evaluation harness, market signals, Hindi. The chat layer is the one that most needs the guardrail already built — which it is.

THEN HARDEN, BECAUSE STRANGERS ARE NEXT

Phase eleven's security and privacy work — the consents dashboard, crypto-shredded erasure, dependency scanning, a formal DPDP alignment pass — should land before anyone outside the project trusts it with real money.

THEN MEASURE INSTEAD OF ASSERTING

Phase twelve's evaluation datasets turn "the classifier is good" into a number against a held-out set, with thresholds that block a merge when they regress. Until that exists, model quality is an opinion.

AND IN PARALLEL, THE NON-ENGINEERING PATH

The funding decision. The legal opinion. Twenty households for sixty days. A visual identity and a store listing. None of these depends on the next engine, and all of them gate a release.

The engineering is the part of this project that is going well. The decisions on the previous pages are the part that is waiting.

How to pick this up

If this landed on your desk tomorrow

IN THIRTY MINUTES

Read the binding rules file at the repository root. Read the flow document for one feature. Install the debug build, tap "Explore with sample data", and use the Purchase Advisor and the order-of-operations screen — they are the product in miniature. The companion document to this one walks every screen.

IN A DAY

Run the full gate and watch it pass. Open one engine: its documentation, its interface, its tests, its golden file and its Python oracle side by side. Then read the decision record for the issue that built it and the session file from that day. That loop — contract, tests, oracle, decision, session — is the whole method in one engine.

IN A WEEK

Ship the next issue by the written sequence, including the parts you will want to skip. Break your own gates on purpose. Write the oracle. Run it on a device. Write the records. The sequence is not ceremony; every step in it has caught something real in this codebase.

WHAT NOT TO DO

Do not let a model produce a number. Do not hardcode a threshold. Do not add a network call to a core path. Do not close an issue on a green build without looking at the screen. Everything else is negotiable; those four are what the project is.

The code is the easy half to inherit. The reason the code is like this is the half that is written down.